

TP5 -- Projet Final :

Application console en C ou C++

Matiere : Decouverte des langages compiles

Niveau : Bachelor 1

Duree : 4h (lancement en classe) + travail personnel

Jour : 5/5 — lundi 15 juin 2026

Evaluation : Projet note (modalite officielle de la matiere)

Contexte et objectifs

Ce projet final evalue l'ensemble des competences acquises lors des TP1 a TP4. Vous devez concevoir et implementer une **application console en C ou C++**, en mobilisant :

- La compilation et les outils de developpement (gcc/g++, Makefile)
- Les types, variables, operateurs et entrees/sorties
- Les structures de controle (if, switch, for, while)
- Les fonctions et la modularisation
- Les tableaux, chaines et pointeurs
- L'allocation dynamique (malloc/free ou new/delete)
- **Bonus C++ :** les classes et l'encapsulation

Choix du projet

Choisissez **un sujet** parmi les propositions suivantes ou proposez le votre (a valider avec le formateur en fin de journee).

Sujet A -- Gestion de bibliotheque

Application de gestion de livres avec CRUD (Creer, Lire, Modifier, Supprimer).

Donnees d'un livre :

- Titre (chaine)
- Auteur (chaine)
- Annee de publication (entier)
- Disponible ou emprunte (booleen)

Fonctionnalites requises :

1. Ajouter un livre
2. Afficher tous les livres
3. Rechercher un livre par titre ou auteur (recherche partielle)
4. Emprunter / rendre un livre (change le statut)
5. Supprimer un livre
6. Sauvegarder dans un fichier texte et recharger au demarrage

Menu :

```
=== BIBLIOTHEQUE ===  
1. Ajouter un livre  
2. Afficher tous les livres  
3. Rechercher  
4. Emprunter / Rendre  
5. Supprimer un livre  
6. Sauvegarder et quitter
```

Sujet B -- Pendu / Mastermind

Jeu en console avec lecture de mots dans un fichier.

Pendu :

- Charger une liste de mots depuis un fichier `mots.txt`
- Tirer un mot au hasard
- L'utilisateur propose des lettres une par une
- Afficher le mot partiellement revele et le nombre d'erreurs restantes
- Fin de partie quand le mot est trouve ou que 7 erreurs sont atteintes
- Gerer le score (nombre de parties gagnees / jouees)

Mastermind :

- L'ordinateur genere un code secret de 4 chiffres (1-6)
 - Le joueur propose des combinaisons
 - Afficher le nombre de chiffres bien places et mal places
 - Maximum 10 tentatives
 - Proposer plusieurs niveaux de difficulte
-

Sujet C -- Carnet d'adresses

Application de gestion de contacts.

Donnees d'un contact :

- Prenom, Nom (chaines)
- Telephone (chaine)
- Email (chaine)

Fonctionnalites requises :

1. Ajouter un contact
2. Afficher tous les contacts
3. Rechercher par nom (insensible a la casse)
4. Modifier un contact existant
5. Supprimer un contact
6. Sauvegarde automatique dans `contacts.csv`

Sujet D -- Gestionnaire de taches (To-Do List)

Application de gestion de taches avec priorites.

Donnees d'une tache :

- Titre (chaîne)
- Priorite (Haute, Normale, Basse)
- Statut (A faire, En cours, Termine)
- Date de creation (chaîne au format "JJ/MM/AAAA")

Fonctionnalites requises :

1. Ajouter une tache
2. Afficher toutes les taches
3. Filtrer par statut ou par priorite
4. Modifier le statut d'une tache
5. Supprimer une tache
6. Sauvegarde dans un fichier texte

Sujet E -- Calculatrice scientifique

Evaluation d'expressions mathematiques.

Fonctionnalites requises :

1. Operations de base (+, -, *, /)
 2. Fonctions : puissance, racine carree, modulo
 3. Historique des calculs (stocke en memoire)
 4. Mode interactif (boucle de saisie)
 5. Gestion des erreurs (division par zero, saisie invalide)
 6. Sauvegarde de l'historique dans un fichier
-

Sujet libre

Vous pouvez proposer votre propre sujet. Exemples :

- Mini base de données clés-valeurs (type Redis simplifiée)
- Convertisseur d'unités multi-domaines
- Jeu du serpent (Snake) en console avec ncurses

Le sujet doit être valide par le formateur en fin de journée.

Structure de fichiers attendue

Votre projet doit être **organisé en plusieurs fichiers** :

```
projet_final/  
|-- Makefile  
|-- README.md      (description, comment compiler, comment utiliser)  
|-- main.c (ou main.cpp)  
|-- module1.c + module1.h  
|-- module2.c + module2.h  
|-- data/  
    |-- (fichiers de sauvegarde)
```

Makefile minimal

Pour un projet C :

```

CC      = gcc
CFLAGS  = -Wall -Wextra -g -std=c11

SRCS = main.c module1.c module2.c
OBJS = $(SRCS:.c=.o)
TARGET = app

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm -f $(OBJS) $(TARGET)

.PHONY: all clean

```

Pour un projet C++ :

```

CXX      = g++
CXXFLAGS = -Wall -Wextra -g -std=c++17

SRCS = main.cpp module1.cpp module2.cpp
OBJS = $(SRCS:.cpp=.o)
TARGET = app

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CXX) $(CXXFLAGS) -o $@ $^

%.o: %.cpp
    $(CXX) $(CXXFLAGS) -c -o $@ $<

clean:
    rm -f $(OBJS) $(TARGET)

.PHONY: all clean

```

Important : Les indentations dans les regles doivent etre des **TABULATIONS**, pas des espaces.

Etapes recommandees du projet

Etape	Description	Duree estimee
1. Cahier des charges	Lister les fonctionnalites, definir les structures de donnees	Jour 5
2. Squelette + Makefile	Creer la structure de fichiers, un main minimal qui compile	Jour 5
3. Implementation	Coder fonction par fonction, tester au fur et a mesure, commiter	5-7 jours
4. Tests	Tester tous les cas (saisies invalides, fichiers manquants)	1 jour
5. README + demo	Documentation d'usage, preparation de la presentation orale	1 jour

Atelier du jour : demarrer son projet

Etapes a realiser aujourd'hui

1. **Choisir** le sujet (parmi la liste ou le votre) et le valider avec le formateur
2. **Creer** un depot Git (GitHub ou GitLab) et l'initialiser avec un README minimal

3. **Rediger** un cahier des charges court : objectifs, fonctionnalites principales, structures de donnees
4. **Creer** la structure de fichiers : main.c/cpp, headers, Makefile minimal
5. **Faire** un premier commit. Verifier que `make` compile sans warning

Astuce : Profitez de cette derniere seance pour me poser toutes vos questions -- c'est le bon moment pour valider votre architecture.

Criteres d'evaluation

Grille de notation (sur 20)

Critere	Points	Ce qu'on regarde
Compilation propre	2	Compile sans warning avec -Wall -Wextra. Makefile fonctionnel.
Fonctionnalite	6	Le programme fait ce qui est demande, gere les cas d'erreur.
Qualite du code	5	Lisibilite, nommage, fonctions courtes, commentaires utiles.
Gestion memoire	3	Pas de fuites memoire, pas de debordements.
Documentation	2	README clair : comment compiler, comment utiliser.
Presentation orale	2	Capacite a expliquer son code et ses choix techniques.
Total	20	

Bonus (jusqu'a +3 points)

- Usage de C++ avec classes et encapsulation (+1)
- Interface utilisateur soignée (menus, tableaux ASCII) (+1)
- Gestion robuste des erreurs (inputs invalides, fichiers manquants) (+1)

Rendu

Format : Archive ZIP contenant le dossier projet **Nom du fichier :**

`NOM_Prenom_ProjetFinal_LangagesCompiles.zip` **Delai :** 2 semaines

après le dernier jour de cours (29 juin 2026)

Présentation orale (10 min) :

1. Démonstration du programme (5 min)
2. Explication d'un choix technique (2 min)
3. Questions du formateur (3 min)

Conseils

- **Commencez par le modèle de données :** définissez vos structures/classes avant d'écrire le main
 - **Compilez souvent :** ne laissez pas s'accumuler les erreurs
 - **Testez chaque fonctionnalité** au fur et à mesure
 - **Gérez les cas limites :** que se passe-t-il si la liste est vide ? Si le fichier n'existe pas ?
 - **Utilisez** `-Wall -Wextra` et corrigez tous les warnings
 - **Commitez régulièrement** avec des messages clairs
-

